

Guida operativa completa al progetto di Distributed Systems 2025-2026

A Quorum-Based Total Order Broadcast Protocol for Distributed Intelligence Databases

Documento di sintesi e progettazione, in italiano, costruito a partire dalla specifica (`ds1_project_2026.pdf`), dalle slide (`ds1_project_2026_presentation.pdf`) e dal template LaTeX fornito. Obiettivo: spiegare in modo esaustivo **che cosa devi fare, perché, come deve essere fatto e quali insidie evitare**, in modo che tu possa attaccare l'implementazione con un piano chiaro e arrivare alla discussione preparato.

1. Il problema in una pagina

Un servizio di intelligence vuole tenere traccia della posizione geografica di un insieme **fisso** di "persone di interesse". Ogni posizione è codificata come un `(int)`, e l'intero stato del sistema è un **array di interi** `P[]`, dove l'indice identifica la persona e il valore è la sua posizione.

Per ragioni di sicurezza, tolleranza ai guasti e disponibilità globale, **il dato non è centralizzato**: è replicato su `(N)` nodi distribuiti geograficamente. I client possono contattare **una qualsiasi replica** per:

- **leggere** un valore: `read(idx) → P[idx]`
- **scrivere** un valore: `write(idx, val)` che imposta `P[idx] = val`

Il tuo compito è progettare e implementare un **protocollo distribuito** che coordini queste repliche garantendo:

1. **Tolleranza ai guasti** tramite **quorum** (`(|Q| = ⌊N/2⌋ + 1)`, maggioranza stretta).
2. **Total Order Broadcast (TOB)**: tutte le scritture sono applicate da tutte le repliche corrette **nello stesso ordine totale**.
3. **Coordinatore distinto** che esegue il protocollo di update; se cade, le repliche **eleggono** un nuovo coordinatore su un **anello**.
4. **Consistenza sequenziale** dal punto di vista di un client che parla sempre con la stessa replica.
5. **Uniform Agreement**: se *anche una sola* replica (eventualmente non corretta) applica un update, **tutte** le corrette finiranno per applicarlo.

Vincoli implementativi: **Akka** (Java), nodi = attori Akka, canali **affidabili e FIFO**, ritardi di rete simulati con delay casuali controllati, almeno la maggioranza delle repliche resta sempre corretta, le repliche crashate **non si riprendono**.

2. Architettura logica del sistema

2.1 Attori in gioco

Attore	Quantità	Ruolo
Replica	<code>N</code> (fissato a init)	Conserva una copia locale di <code>P[]</code> . Risponde a letture, inoltra scritture al coordinatore, partecipa al TOB e alle elezioni.
Coordinatore	1 (è sempre una delle <code>N</code> repliche)	Avvia il protocollo a due fasi per ogni update. Emette <code>HEARTBEAT</code> .
Client	qualunque, esterni	Inviano <code>READ</code> e <code>WRITE</code> request a una replica a piacere.

Nota fondamentale: **il coordinatore è una replica**. Quando si calcola il quorum, il coordinatore conta come uno dei nodi del quorum (lo dicono esplicitamente le slide).

2.2 Stato interno di una replica

Ogni replica deve mantenere almeno:

- `int[] P` — la lista delle posizioni (dimensione fissa, decisa a inizializzazione).
- `int epoch` — epoca corrente `e` (cambia ad ogni nuova elezione).
- `int seqNumber` — sequence `i` dell'ultimo update; **si resetta a 0** ad ogni nuova epoca.
- `History updateHistory` — storico degli update osservati, indicizzato dalla coppia `<e, i>`.
- `ActorRef currentCoordinator` — riferimento al coordinatore noto in questo momento.
- `List<ActorRef> replicas` — l'intera membership (statica, definita a init).
- Tabelle di **timer** (`Cancellable`) per i vari timeout (vedi §6.5).
- Buffer per ACK ricevuti per ciascun update pending (solo se sei coordinatore).
- Stato di partecipazione all'elezione (vedi §7).

Il template suggerisce di creare:

- una classe `UpdateID` (o simile) immutabile per la coppia `<epoch, seqNumber>` con `equals`, `hashCode`, `compareTo` (così la puoi usare come chiave in `Map`).
- una classe `Update` immutabile che incapsula `<UpdateID, idx, val>`.
- una `UpdateHistory` con metodi `add`, `getMostRecent`, `getMissing(since)`, `contains(updateID)`.

Incapsulamento Akka: lo stato di un attore è **strettamente privato**. Niente variabili statiche condivise, niente collezioni mutabili passate per riferimento ad altri attori. Se devi davvero condividere un oggetto, deve essere **immutabile** (specifica esplicita del prof).

3. Tipologie di messaggi (schema dei tipi)

Tutti i messaggi sono **classi immutabili** (campi `final`), costruttori che fanno copie difensive di liste/array). Suggerisco di organizzarli in un package `messages` con sotto-package per fase:

```
messages/
  client/
    ReadRequest      { ActorRef client; int idx; }
    ReadResponse     { boolean success; int idx; int val; }
    WriteRequest     { ActorRef client; int idx; int val; }
    WriteResponse    { boolean success; int idx; int val; }
  protocol/
    Update           { UpdateID id; int idx; int val; }          // coord → repliche
    Ack              { UpdateID id; ActorRef sender; }           // repliche → coord
    WriteOk          { UpdateID id; }                            // coord → repliche
    Heartbeat        { /* eventuale epoch */ }                  // coord →
  repliche
    election/
      Election        { Map<ActorRef, UpdateID> candidates; }    // gira sull'anello
      ElectionAck      { UpdateID lastKnown; }                   // ack hop-by-hop
      Synchronization { ActorRef newCoordinator; int newEpoch; List<Update>
missing; }
    control/
      UpdateTimeout   { /* timer su WRITEOK */ }
      ForwardTimeout  { /* timer su UPDATE atteso dal coord */ }
      HeartbeatTimeout { /* coord muto da troppo tempo */ }
      ElectionTimeout { /* anello non termina */ }
      CrashCommand    { CrashPoint where; }                     // per testing:
vedi §9
```

Tutti i tipi devono essere `Serializable` (Akka requirement), e tutti i campi `final`. I `Map`/`List` interni vanno avvolti in `Collections.unmodifiableX(...)` o in copie difensive.

4. Il protocollo di update (Two-Phase Broadcast)

È il cuore del progetto. Ogni `write(idx, val)` percorre questo flusso:

4.1 Fase 0 — Client → Replica

1. Il client manda `WriteRequest(client, idx, val)` a una replica qualunque `R`.
2. `R` logga: `[Client <name>] requesting WRITE (<idx>,<val>) to <ReplicaID>`.
3. Se `R` è il coordinatore, salta direttamente alla fase 1.
4. Altrimenti `R` inoltra la richiesta al `currentCoordinator` e avvia `ForwardTimeout`: se non vede arrivare l'`Update` corrispondente in tempo, deduce che il coordinatore è morto → fa partire l'elezione.

4.2 Fase 1 — Coordinatore broadcast UPDATE

5. Il coordinatore (C) riceve la `WriteRequest` (direttamente o inoltrata).
6. (C) incrementa il proprio `seqNumber` (`i := i + 1`) nell'epoca corrente (e).
7. (C) crea un `Update` con `id = <e, i>` e lo **broadcasta a TUTTE le repliche, incluso sé stesso**.
8. Inizializza un contatore di ACK per `<e, i>` (oppure un `Set<ActorRef>` di chi ha ackato).

4.3 Fase 2 — Repliche rispondono ACK

9. Ogni replica (R) riceve `Update(<e,i>, idx, val)`:
 - lo memorizza in `updateHistory` come **"pending"** (NON applicato ancora);
 - manda `Ack(<e,i>)` al coordinatore;
 - **avvia** `UpdateTimeout` per `<e,i>`: se non riceve `WriteOk` entro X ms → coordinatore morto → elezione.

4.4 Fase 3 — Coordinatore raggiunge il quorum

10. (C) raccoglie gli ACK. Appena conta `Q = ⌊N/2⌋ + 1` ACK distinti per `<e,i>` (sé stesso compreso, perché conta come uno):
 - broadcast di `WriteOk(<e,i>)` a tutte le repliche;
 - applica l'update localmente: `P[idx] = val`;
 - logga: `[Replica <CID>] applied update <e>:<i> (<idx>,<val>)`.

4.5 Fase 4 — Repliche applicano

11. Ogni replica che riceve `WriteOk(<e,i>)`:
 - cerca `<e,i>` nella `updateHistory` (deve esserci come pending);
 - **applica** `P[idx] = val`;
 - cancella `UpdateTimeout` associato;
 - logga: `[Replica <RID>] applied update <e>:<i> (<idx>,<val>)`.

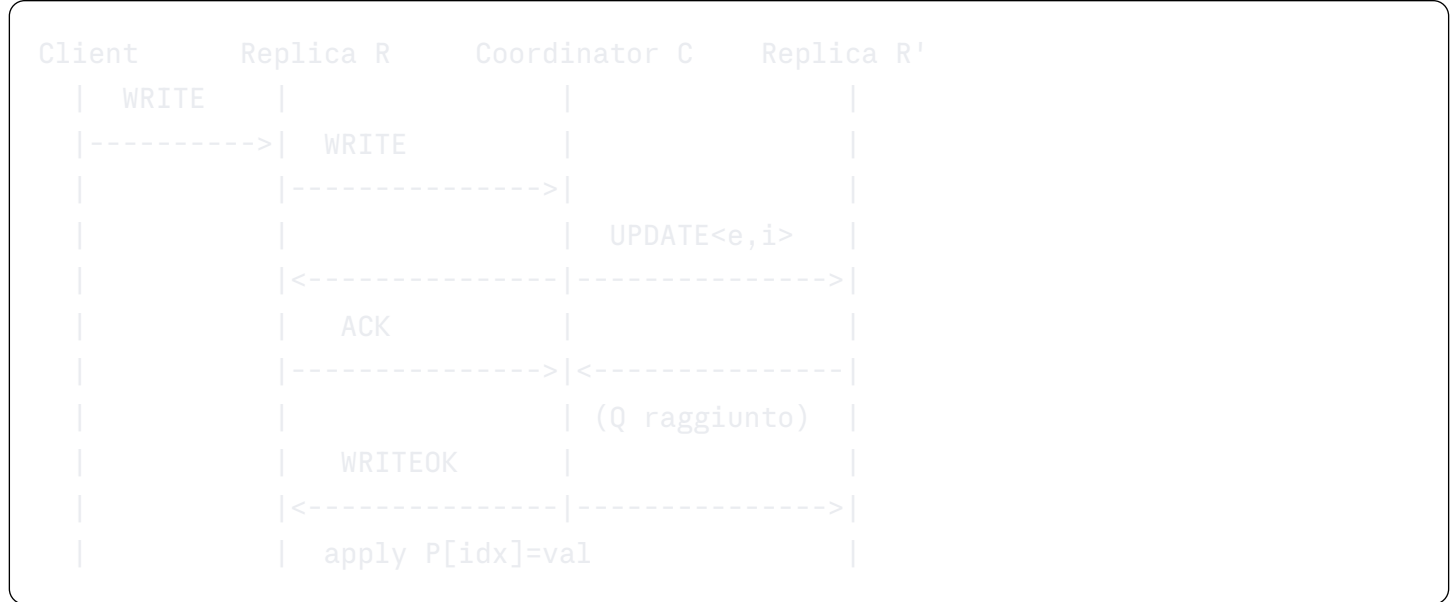
4.6 Total Order: perché funziona

Grazie a canali FIFO e al fatto che **un unico coordinatore** è la sola sorgente di `Update` e `WriteOk`:

- Tutti i nodi vedono gli `Update` nello stesso ordine in cui li ha emessi il coordinatore.
- Tutti i nodi vedono i `WriteOk` nello stesso ordine.
- Quindi tutti applicano gli update nello stesso ordine.

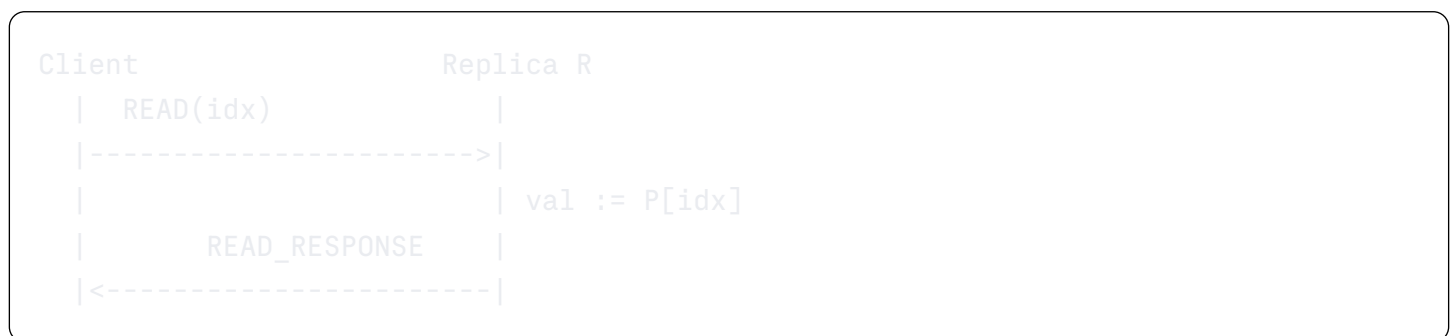
Insidia da non sottovalutare: l'ordine totale vale **all'interno di un'epoca**. Cambio di epoca = cambio di coordinatore. Tra epoche, l'ordine è garantito perché (e) è strettamente crescente e l'elezione recupera gli update incompleti dell'epoca precedente (uniform agreement).

4.7 Diagramma temporale (testuale)



5. Letture (read requests)

Le letture **non passano per il coordinatore**: la replica contattata risponde con il valore corrente che ha localmente in `P[idx]`.



Consistenza sequenziale per-replica: tutte le operazioni emesse da un client verso una specifica replica vengono osservate nell'ordine in cui sono state inviate. Questa garanzia segue automaticamente da:

- FIFO sui canali (Akka per-receiver è FIFO se non usi `tell` da thread diversi a casaccio);
- il fatto che la replica processa i messaggi in ordine di arrivo nel proprio mailbox.

Sottigliezza: se il client manda `WRITE` poi `READ` alla stessa replica, la `READ` potrebbe vedere un valore *vecchio*, perché la `WRITE` deve fare il giro del coordinatore prima di essere applicata. Questa è una **proprietà del modello**, non un bug. Il prof la chiama "consistenza sequenziale dal punto di vista del client che parla sempre con la stessa replica" e va citata nel report.

Sul client: timeout di lettura. Se la replica non risponde entro X ms, logga `TIMEOUT READ ...`.

6. Crash detection (rilevamento dei guasti)

6.1 Modello di guasto

- Solo **crash-stop** (i nodi si fermano e non riemergono).
- **Almeno una maggioranza** di repliche rimane sempre corretta ($f \leq \lfloor N/2 \rfloor$).
- Rilevamento **accurato**: niente falsi positivi → puoi scegliere timeout "generosi".

6.2 Tre tipi di timeout sul coordinatore

Le slide elencano esattamente tre situazioni in cui una replica deduce che il coordinatore è morto:

Trigger	Timer	Quando si avvia	Azione al fire
<code>WRITEOK</code> mancante	<code>UpdateTimeout</code>	quando ricevo <code>Update<e, i></code>	inizia elezione
<code>UPDATE</code> mancante	<code>ForwardTimeout</code>	quando ho inoltrato una <code>WriteRequest</code> al coord	inizia elezione
silenzio prolungato	<code>HeartbeatTimeout</code>	sempre, resettato ad ogni <code>Heartbeat</code> o messaggio dal coord	inizia elezione

6.3 Heartbeat

Il coordinatore broadcasta periodicamente un messaggio `Heartbeat`. Periodo suggerito: 1 s. Ogni replica resetta `HeartbeatTimeout` ($\approx 3 \times$ periodo, p.es. 3 s) ad ogni messaggio ricevuto dal coordinatore (non solo i heartbeat: anche `Update`/`WriteOk` valgono come segno di vita).

6.4 Scelta dei valori di timeout (importante per il report)

I valori devono essere **abbastanza grandi da non scattare per via dei delay simulati di rete**, ma abbastanza piccoli da rendere il sistema reattivo. Una taratura ragionevole:

- delay di rete simulato: 10-50 ms uniforme;
- `UpdateTimeout` ≈ 1.5 s;
- `ForwardTimeout` ≈ 1.5 s;
- `Heartbeat` ogni 1.0 s, `HeartbeatTimeout` ≈ 3.0 s;
- `ElectionAck` timeout ≈ 1.0 s;
- `Election` complessivo (vedi §7.4) $\approx 5-8$ s.

Documenta questi valori e la loro motivazione nel report.

6.5 Gestione corretta dei timer in Akka

```

java

import akka.actor.Cancellable;
import java.time.Duration;

private Cancellable scheduleTimeout(Object msg, long ms) {
    return getContext().getSystem().scheduler().scheduleOnce(
        Duration.ofMillis(ms),
        getSelf(),           // destinatario
        msg,                  // messaggio inviato a sé stesso al fire
        getContext().getDispatcher(),
        getSelf()             // sender = self
    );
}

private void cancelTimeout(Cancellable c) {
    if (c != null) c.cancel();
}

```

Trappola sottolineata dalle slide (Implementation suggestions): se rischeduli un timer **sulla stessa variabile** senza prima cancellare il precedente, `.cancel()` **non riuscirà più a fermare quello vecchio**. Quindi:

```

java

private Cancellable updateTimer;

void rearmUpdateTimer() {
    cancelTimeout(updateTimer);           // PRIMA cancella
    updateTimer = scheduleTimeout(new UpdateTimeout(...), 1500);
}

```

Per timer "per-update", una `Map<UpdateID, Cancellable>` è più pulita di una singola variabile.

7. Elezione del coordinatore (ring-based)

7.1 Definizione dell'anello

L'anello è definito dall'**ordine crescente degli ID delle repliche**. Esempio con 5 nodi: `0 → 1 → 2 → 3 → 4 → 0`. Il "successivo" si calcola al volo:

```

java

ActorRef nextInRing(int myId) {
    int next = (myId + 1) % N;
    while (replicas.get(next).isKnownDead()) next = (next + 1) % N;
    return replicas.get(next);
}

```

In realtà non sai chi è morto in modo "globale": lo scopri solo se l'ACK non arriva (vedi §7.5).

7.2 Avvio dell'elezione

Una replica (R) entra nello stato (ELECTION) quando rileva il crash del coordinatore (uno qualsiasi dei tre timeout di §6.2 scatta) **e non sta già partecipando**. Crea un messaggio:

```
Election { candidates: { R: lastKnownUpdate(R) } }
```

e lo manda al successivo nell'anello, armando un (ElectionAckTimeout).

7.3 Ricezione di ELECTION

Una replica (R') che riceve (Election):

1. **Manda ACK al mittente** ((ElectionAck)) — questo serve a chi le ha mandato il messaggio per sapere che lei è viva.
2. Se (R') **non è già** in stato (ELECTION):
 - aggiunge sé stessa al (Map<ActorRef, UpdateID>) di candidati;
 - passa in stato (ELECTION) (vedi §8.2 sulla (Receive) separata);
 - inoltra al successivo nell'anello.
3. Se (R') **è già** in stato (ELECTION) e nel messaggio **vede già il proprio ID**:
 - significa che il messaggio ha fatto un giro completo → **decide il vincitore**:
 - sceglie il candidato con (UpdateID) massimo;
 - in caso di pareggio, **id replica più alto** (regola fissata dal prof).
4. Se è già in elezione ma il proprio ID non è nei candidati (caso raro di seconda ondata dovuta a crash):
 - aggiunge/aggiorna comunque la propria info e inoltra.

7.4 Annuncio del vincitore: SYNCHRONIZATION

Quando un nodo si dichiara vincitore (è "il migliore" e ha visto un giro completo):

1. Diventa il nuovo coordinatore.
2. **Apri una nuova epoca**: (epoch := epoch + 1), (seqNumber := 0).
3. Esamina la propria (updateHistory) e ricostruisce eventuali update pending ((<e_prev, i>) mai confermati con (WriteOk) ma presenti come pending nella sua storia).
4. **Completa gli update interrotti**: per ognuno di essi, manda subito un (WriteOk(<e_prev, i>)) a tutti, e li applica. Questo è ciò che garantisce **Uniform Agreement** (Hint 3 delle slide).
5. Broadcast (Synchronization(newCoord = self, newEpoch = e+1, missing = [...])).

Le altre repliche, ricevendo (Synchronization):

- aggiornano `currentCoordinator` e `epoch`;
- applicano tutti gli `missing` (ordinate per `<e, i>`);
- escono dallo stato `ELECTION` e tornano in `NORMAL`;
- cancellano i loro timer di elezione.

7.5 Crash durante l'elezione

Se chi forwarda un `Election` non riceve `ElectionAck` entro il timeout:

- assume che il successivo nell'anello sia morto;
- **lo salta** e manda al successivo del successivo;
- riavvia il timer.

Si assume che **almeno una maggioranza** sia ancora viva, e che almeno una di queste contenga "l'update più recente" — quindi l'elezione converge.

7.6 Terminazione dell'elezione (Hint 2 delle slide)

Caso patologico: il candidato vincente crasha **prima** di annunciarsi. Il messaggio `Election` continuerebbe a girare per sempre. Soluzione:

- ogni nodo che entra in elezione arma anche un `GlobalElectionTimeout` (lungo, p.es. 6 s);
- allo scadere, **riavvia da capo l'elezione** (nuovo `Election` con sé stesso come unico candidato iniziale).

Questo trasforma una potenziale livelock in qualcosa che converge.

7.7 "Most recent update does not change" (Hint 1 delle slide)

Mentre un nodo è in stato `ELECTION`, **non deve cambiare** il proprio "last known update", altrimenti il confronto fra candidati ballerebbe. Conseguenza pratica:

- nello stato `ELECTION`, una replica **non processa** `Update`/`WriteOk` **di nuova generazione**: li può accodare/scartare. Il nuovo coordinatore farà partire da capo la consegna con `Synchronization` + nuova epoca.

8. Macchina a stati di una replica

8.1 Stati principali

Stato	Significato
NORMAL	Funzionamento ordinario: gestisce Read/Write/Update/WriteOk/Heartbeat.
ELECTION	Sta partecipando a un'elezione. Ignora gli Update/WriteOk di nuova generazione, gestisce Election/Sync.
CRASHED	È crashata. Ignora tutto, non manda niente.

In Akka questo si traduce in **più** Receive **distinte** (Implementation suggestion 2 delle slide).
Esempio:

java

```
private Receive normalReceive() {
    return receiveBuilder()
        .match(ReadRequest.class, this::onRead)
        .match(WriteRequest.class, this::onWrite)
        .match(Update.class, this::onUpdate)
        .match(Ack.class, this::onAck)
        .match(WriteOk.class, this::onWriteOk)
        .match(Heartbeat.class, this::onHeartbeat)
        .match(UpdateTimeout.class, this::onUpdateTimeout)
        .match(ForwardTimeout.class, this::onForwardTimeout)
        .match(HeartbeatTimeout.class, this::onHeartbeatTimeout)
        .match(CrashCommand.class, this::onCrashCommand)
        .build();
}

private Receive electionReceive() {
    return receiveBuilder()
        .match(Election.class, this::onElection)
        .match(ElectionAck.class, this::onElectionAck)
        .match(Synchronization.class, this::onSync)
        .match(ElectionTimeout.class, this::onElectionTimeout)
        .match(CrashCommand.class, this::onCrashCommand)
        // ignora silenziosamente Update/WriteOk vecchi
        .matchAny(m -> {})
        .build();
}

private Receive crashedReceive() {
    return receiveBuilder()
        .matchAny(m -> { /* drop everything */ })
        .build();
}

// transizione:
getContext().become(electionReceive());
```

8.2 Stato **CRASHED** (vedere §9)

java

```
private void crash() {
    log("CRASHED");
    getContext().become(crashedReceive());
    // NON cancellare la mailbox; semplicemente non rispondere più
}
```

Importante: **non fare** `getContext().stop(getSelf())`. L'attore deve continuare a esistere per "raccogliere" silenziosamente i messaggi che gli arrivano, semplicemente non risponde più. Altrimenti chi gli manda messaggi vedrebbe `DeadLetters`, che è diverso dalla semantica di "nodo crashato che ignora".

9. Crash controllati per i test

Il prof richiede che, con "minimal instrumentation", si possano **forzare crash in punti specifici**:

- durante il broadcast di `UPDATE` (es. dopo aver mandato a 2 su 4 destinatari);
- dopo aver ricevuto `UPDATE` ma prima di mandare `ACK`;
- durante la dissemination di `WRITEOK` (dopo aver mandato `WRITEOK` ad alcuni e prima di averlo mandato a tutti);
- durante l'elezione (mentre tiene un `Election` in mano).

Pattern: introdurre un `enum CrashPoint` e un campo `pendingCrash` che ogni handler controlla in punti precisi:

```
java

enum CrashPoint {
    NONE,
    BEFORE_SENDING_UPDATE,
    DURING_UPDATE_BROADCAST,
    AFTER_RECEIVING_UPDATE_BEFORE_ACK,
    AFTER_SENDING_ACK,
    DURING_WRITEOK_BROADCAST,
    AFTER_FIRST_WRITEOK,
    DURING_ELECTION_FORWARDING,
    AFTER_ELECTION_WINNER
}

private void maybeCrash(CrashPoint point) {
    if (pendingCrash == point) crash();
}
```

Quindi un test fa qualcosa tipo:

```
java

coord.tell(new CrashCommand(CrashPoint.DURING_WRITEOK_BROADCAST), ActorRef.noSender()
// poi invia una write
```

E il coordinatore, dentro il loop di broadcast di `WriteOk`, dopo aver mandato al primo destinatario chiama `maybeCrash(DURING_WRITEOK_BROADCAST)` e si auto-uccide se è il punto programmato.

10. Delay di rete simulati

Le slide e la specifica chiedono **delay casuali ma controllati** fra l'invio di due messaggi consecutivi (per emulare propagazione di rete). Pattern standard mostrato in classe:

```
java

private void delaySend(ActorRef to, Object msg) {
    try {
        Thread.sleep(rnd.nextInt(MAX_DELAY_MS));
    } catch (InterruptedException ignored) {}
    to.tell(msg, getSelf());
}
```

Attenzione: `Thread.sleep` dentro un attore **blocca il dispatcher**. È accettato in questo progetto (lo fanno gli esempi del corso), ma:

- usa valori piccoli (`MAX_DELAY_MS ≤ 50`);
- non farlo in `preStart`/`preRestart`;
- nei test automatici fa schifo: assicurati che il framework dei test sopporti un po' di sleep.

Alternativa più "pulita" ma più rumorosa: `scheduler.scheduleOnce(delay, () -> to.tell(msg, self))`. Va benissimo, e non blocca il dispatcher.

11. Logging

Le slide e la specifica richiedono **timestamp con precisione millisecondo** in testa a ogni log. La codebase fornirà funzioni di utilità: **usale**, perché stampe ad hoc rompono i test automatici.

Formato atteso (esempio):

```
12:34:56.789 [Client alice] requesting WRITE (3, 42) to Replica2
12:34:56.812 [Replica 2] applied update 1:7 (3, 42)
```

Le righe **minime** richieste sono:

- request/complete/timeout di READ e WRITE lato client;
- applied update lato replica;
- replica crashed.

Aggiungine altre (con sobrietà) per: ricezione `UPDATE`, invio `ACK`, raggiunto quorum, broadcast `WRITEOK`, inizio elezione, `ELECTION` ricevuto/inoltrato, vincitore eletto, `SYNCHRONIZATION` ricevuta. Sono utilissime in fase di demo.

12. Tabella delle invarianti (da citare nel report)

Invariante	Significato	Garantita da
TO-1	Se due repliche corrette applicano <code>U</code> e <code>U'</code> , le applicano nello stesso ordine.	Unico coordinatore per epoca + FIFO + <code><e, i></code> monotone.
UA-1 (Uniform Agreement)	Se anche solo una replica applica <code>U</code> , tutte le corrette lo applicheranno.	Quorum + recovery degli incomplete update nel nuovo coordinatore.
Integrity	<code>U</code> consegnato al massimo una volta da ogni replica.	<code>updateHistory</code> indicizzata da <code><e, i></code> .
Validity	Una <code>WriteRequest</code> di un client corretto verso una replica corretta viene eventualmente applicata.	Quorum sempre disponibile + elezione termina.
No-loss durante elezione	Update incompleti dell'epoca morente vengono recuperati.	Step "complete pending updates" nel nuovo coordinatore.
No falsi positivi	Una replica viva non viene mai considerata morta.	Timeout sufficientemente grandi rispetto ai delay simulati.

13. Scaletta di sviluppo consigliata (in ordine)

Le slide stesse suggeriscono un ordine — eccolo espanso:

Sprint 1 — "Happy path"

1. Clona la codebase del prof, leggi il `README` e gli esempi di test.
2. Modella le classi `UpdateID`, `Update`, `UpdateHistory` e tutti i messaggi.
3. Implementa la replica (senza crash) con la `Receive` `NORMAL`:
 - read request → risposta diretta;
 - write request → inoltro al coordinatore;
 - update/ack/writeok del protocollo a 2 fasi;
4. Implementa il client base (sequenze hard-coded di read/write).
5. Esegui localmente: 3 repliche, scrivi, leggi, controlla i log.
6. Lancia i test automatici della codebase su questo subset.

Sprint 2 — Heartbeat + crash semplici

7. Aggiungi `Heartbeat` periodico dal coordinatore.
8. Implementa `CRASHED` (cambio di Receive a `crashedReceive`).
9. Implementa `CrashCommand` con i `CrashPoint` di base.
10. Aggiungi i tre timeout (`UpdateTimeout`, `ForwardTimeout`, `HeartbeatTimeout`).
11. Test scenario: coordinatore vivo, una replica non-coord crasha → il sistema continua.

Sprint 3 — Elezione

12. Aggiungi lo stato `ELECTION` e la sua `Receive`.
13. Implementa il giro dell'anello con `Election`+`ElectionAck` hop-by-hop.
14. Implementa la scelta del vincitore (max `<e, i>`, tie-break su id).
15. Implementa `Synchronization` con propagazione di `missing` updates.
16. Test scenario: coordinatore crasha → nuova elezione → riprendi a scrivere.

Sprint 4 — Casi corner

17. Crash del coordinatore **durante** il broadcast di `UPDATE`: assicurati che la nuova epoca riproponga gli update mancanti.
18. Crash del coordinatore **dopo aver mandato** `WRITEOK` **ad alcuni**: assicurati che chi non lo ha visto lo applichi via `Synchronization`. (**Questo è il caso più delicato e quasi sicuro alla discussione.**)
19. Crash di due nodi consecutivi nell'anello durante un'elezione: `ElectionAckTimeout` deve far saltare l'anello al successivo del successivo.
20. Crash del vincitore dell'elezione prima di mandare `Synchronization`: il `GlobalElectionTimeout` riavvia l'elezione da capo.
21. Verifica che `ELECTION` resista a `Update` di vecchia epoca (li ignora o li tratta come storico).
22. Lancia **tutti** i test automatici della codebase.

Sprint 5 — Report e demo

23. Scrivi il report (vedi §15).
24. Prepara **tre o quattro esempi rappresentativi** di esecuzione con log puliti per la demo. Includi corner case (es. crash di 2 nodi).

14. Cose da NON fare (errori frequenti)

- ✗ Usare **variabili statiche** mutabili condivise fra attori (rompe l'incapsulamento, il prof lo segnala).
 - ✗ Far processare alla `electionReceive` i normali `Update`/`WriteOk` come se nulla fosse: rompi Hint 1.
 - ✗ Schedulare un timer senza annullare il precedente sulla stessa variabile.
 - ✗ Applicare un update due volte (mettere `if (history.alreadyApplied(<e,i>)) return;`)).
 - ✗ Usare il `seqNumber` dell'epoca vecchia in quella nuova: si resetta.
 - ✗ Includere il **coordinatore vecchio** nel calcolo del quorum nell'epoca nuova prima della sincronizzazione: sostituisilo prima.
 - ✗ `stop(self)` per "crashare": deve restare in vita ma muto.
 - ✗ Stampare con `System.out.println`: usa le utility della codebase.
 - ✗ Timeout troppo corti: scattano coi delay simulati e generano falsi positivi.
 - ✗ Pensare che FIFO sia globale: è **per-coppia mittente→ricevente**.
-

15. Struttura del report (3-6 pagine)

Il template fornisce tre file: `01_structure.tex`, `02_design.tex`, `03_implementation.tex`. Le slide elencano le domande tipiche a cui il report deve rispondere:

How do you store updates? How do you manage timeouts? How do you control crashes? What operations does a new coordinator perform? How do you ensure election does not block forever?

15.1 Sezione 1 — Project Structure

- Una panoramica dell'architettura: chi sono gli attori, com'è organizzato il package.
- Le classi-dato principali (`UpdateID`, `Update`, `UpdateHistory`) con un piccolo diagramma.
- L'elenco dei messaggi raggruppati per fase.

15.2 Sezione 2 — System Design

- Diagramma con i tre stati e le transizioni (NORMAL ↔ ELECTION ↔ CRASHED).
- Sequence diagram del protocollo di update in condizioni normali.
- Sequence diagram dell'elezione con almeno un crash nell'anello.
- Discussione delle invarianti (tabella di §12).
- Scelta dei valori dei timeout e perché.

15.3 Sezione 3 — Implementation

- Strutture dati concrete (`Map<UpdateID, Update>`, `Map<UpdateID, Set<ActorRef>>` per gli ACK, ecc.).
- Schema di gestione dei timer (pattern `Map<UpdateID, Cancellable>`).
- Punti di crash (`CrashPoint`) e quali punti sono coperti).
- Come gestisci la sincronizzazione dopo elezione e il recupero degli update incompleti.
- Come hai gestito la terminazione forzata dell'elezione (Hint 2).
- Eventuali **ipotesi extra** rispetto alla specifica (es. una taratura specifica, una sotto-ottimizzazione).

■ Tieniti a **4 pagine**, massimo 5. Il prof rigetta i report sopra le 6.

16. Demo e discussione orale (12 min)

Preparati **tre o quattro scenari** con esecuzione live:

1. **Happy path:** `N=5`, due write, una read, log puliti.
2. **Crash non-coord:** una replica casuale crasha, due write proseguono.
3. **Crash coordinator + elezione "pulita":** il coord crasha mentre il sistema è quiescente, nuova elezione, scrittura nella nuova epoca.
4. **Crash coordinator a metà dell'update + recovery:** l'update viene completato dal nuovo coordinatore (è qui che dimostri Uniform Agreement; aspettati domande).

Per ognuno: prepara uno script o un `main` dedicato, e durante la demo **fai vedere i log dal vivo**, commentandoli passo per passo.

Domande tipiche orali che ti chiederanno con alta probabilità:

- Cosa succede se il coordinatore manda WRITEOK a una sola replica e poi muore? Risposta: quella replica applica, le altre no. Alla prossima elezione, il vincitore avrà nella sua history quell' `<e, i>` come pending **oppure** applicato. Se anche una sola replica corretta lo ha già applicato (o ce l'ha pending), il nuovo coordinatore lo recupera con `Synchronization`. **Uniform Agreement preservata.**
 - Come fai a evitare doppia applicazione dopo Synchronization? Risposta: `updateHistory` indicizzata da `<e, i>`; se l'update è già "applied", lo ignori.
 - Perché il quorum include il coordinatore? Risposta: il coordinatore è una replica, anche lui memorizza l'update. Conta come "voto" perché ha visto il messaggio prima di chiunque.
 - Perché ti basta la maggioranza e non N? Risposta: con $f \leq \lfloor N/2 \rfloor$ guasti, due quorum qualunque si intersecano in almeno un nodo corretto → il nuovo coordinatore vede sempre l'update più recente. (Argomento classico Paxos-style.)
 - Cosa garantisce il total order tra epoche? Risposta: `<e, i>` con `e` strettamente crescente. Update con epoca minore precedono quelli con epoca maggiore, sempre.
 - Cosa fai se due timeout scattano insieme? Risposta: entrambi finiscono nel mailbox. Quando processo il primo, entro in `ELECTION`; il secondo viene gestito dalla `electionReceive` che lo ignora (idempotente). Posso anche guardare un "election started" flag.
 - Cosa succede se un client manda due write in fila molto rapidamente? Risposta: la replica le inoltra in ordine FIFO al coordinatore; il coordinatore le serializza con seq numbers consecutivi; il TOB garantisce che vengano applicate nello stesso ordine ovunque.
-

17. Checklist finale prima della consegna

- ☐ Tutti i test automatici della codebase passano.
 - ☐ Tutti i log richiesti (`requesting`, `complete`, `TIMEOUT`, `applied`, `CRASHED`) sono presenti e nel formato giusto.
 - ☐ Timestamp con precisione ms in testa.
 - ☐ Nessun `System.out.println`.
 - ☐ Nessun `static` mutable condiviso.
 - ☐ Tutti i messaggi sono immutabili e `Serializable`.
 - ☐ I `Cancellable` vengono sempre cancellati prima di essere riassegnati.
 - ☐ Le `Receive` per `NORMAL`, `ELECTION`, `CRASHED` sono separate.
 - ☐ Il coordinatore recupera correttamente gli update incompleti.
 - ☐ L'elezione termina anche se il vincitore crasha (`GlobalElectionTimeout`).
 - ☐ Il report è in inglese, tra 3 e 6 pagine, usa il template fornito.
 - ☐ Archivio `tar -czvf CognomeCognome.tgz CognomeCognome` contenente codice + report.
 - ☐ Email a `gianpietro.picco@unitn.it`, `thomas.pasquali@unitn.it`, `stefano.genetti@unitn.it`.
 - ☐ Slot di presentazione prenotato.
-

18. Riassunto in dieci righe (cheat sheet)

1. N repliche con array $P[]$, una è coordinatore.
 2. Read = risposta locale. Write = al coord, che fa UPDATE → quorum di ACK → WRITEOK.
 3. Ogni update ha id $\langle e, i \rangle$, e cresce con le epoche (cambio coord), i riparte da 0.
 4. Quorum = $\lfloor N/2 \rfloor + 1$. Coordinatore conta nel quorum.
 5. Canali reliable + FIFO ⇒ tutti applicano nello stesso ordine.
 6. Crash detection via tre timeout (WRITEOK, UPDATE-after-forward, Heartbeat).
 7. Coordinator giù → elezione su anello con Election/ElectionAck, vince chi ha l'update più recente (tie-break su id).
 8. Vincitore: **prima recupera gli update incompleti**, poi Synchronization con nuova epoca.
 9. Uniform Agreement = recovery degli update pending all'inizio della nuova epoca.
 10. Stati: NORMAL / ELECTION (ignora update di rete normale) / CRASHED (ignora tutto).
-

Buon lavoro. Se mentre implementi ti accorgi che una scelta della specifica è ambigua, scrivila come "ipotesi" nel report e motivala — la specifica stessa lo richiede.